

Optimizing TinyML with Quantization and Distillation for Indoor Localization in Resource-Constrained Environments

Mueid Islam Arian, Md. Mahamudul Hasan, Md. Nafiz Un Nabi Ishan
Department of Electrical and Computer Engineering
North South University, Dhaka, Bangladesh
Email: {mueid.arian, mahamudul.hasan04, nafiz.ishan}@northsouth.edu

Abstract—Indoor localization is a key enabler for smart buildings, asset tracking, and context-aware systems, but state-of-the-art deep learning models are often too heavy for ultra-low-power microcontrollers. This work investigates Tiny Machine Learning (TinyML) techniques to deploy accurate indoor localization models on resource-constrained edge devices. Using RSSI-like magnetic time-series data from the UJIIndoorLoc-Mag dataset, we design several lightweight Mamba-inspired and Transformer-style sequential models implemented with LSTM layers. We then apply dynamic INT8 quantization and knowledge-distillation-inspired training to reduce model size while preserving performance. Our best non-quantized model (Mamba_H32L1) achieves a test F1 score of 93.38% with only ~ 41 kB, while a quantized Mamba_H16L1 student model attains 67–70% F1 with a memory footprint of about 12 kB, making it suitable for deployment on ESP32-class microcontrollers with 520 kB SRAM. Experiments show that quantization yields $3\text{--}4\times$ compression with less than 1% F1 degradation, and data augmentation improves F1 by 5–8%. We analyse per-class performance, model efficiency (F1-per-kilobyte), and trade-offs between accuracy and footprint, and we discuss societal, environmental, and regulatory impacts of privacy-preserving on-device localization. The results demonstrate the feasibility of deploying deep-learning-based indoor localization systems entirely on low-power edge devices.

Index Terms—TinyML, Indoor Localization, Edge Computing, Quantization, Knowledge Distillation, ESP32

I. INTRODUCTION

GPS has transformed outdoor navigation, but its accuracy degrades severely indoors due to signal attenuation and multipath effects. As a result, indoor localization for smart homes, health monitoring, industrial automation, and energy-efficient buildings must rely on alternative modalities such as Wi-Fi, Bluetooth Low Energy (BLE), inertial sensors, or magnetic field signatures. Many existing systems use deep neural networks for fingerprinting-based localization, but these models are typically deployed on smartphones or cloud servers and require megabytes of memory and high compute budgets.

Recent advances in Tiny Machine Learning (TinyML) seek to deploy neural networks directly on microcontrollers with typically less than 1 MB of flash and a few hundred kilobytes of SRAM. At the same time, sequential architectures such as Transformers and Selective State Space Models (e.g., Mamba) have shown strong performance on temporal data, but naively porting them to microcontrollers remains challenging.

This work addresses the question: *How can we design indoor localization models that fit into tens of kilobytes, run in real time on microcontrollers, and still provide usable accuracy?* We focus on BLE- or magnetometer-like RSSI time-series data and explore a combination of lightweight architectures, dynamic quantization, and distillation-style training to compress models while preserving predictive power.

A. Background and Motivation

Indoor localization is widely used in smart homes, hospitals, warehouses, and office buildings for occupancy monitoring, assistive navigation, and safety applications. However, existing solutions often face one or more of the following limitations:

- Camera-based systems raise privacy concerns and require high compute.
- RFID or infrared infrastructures are expensive to deploy and maintain.
- Classical machine learning models (e.g., k-NN, SVM) require manual feature engineering and often lack robustness to temporal variability in RSSI.

BLE RSSI and magnetic signatures provide a low-cost, non-intrusive way to discriminate indoor locations, but the signals are noisy, affected by multipath, walls, and human movement. Deep learning models such as LSTMs or Transformers can learn sequential patterns directly from data, but their parameter counts often exceed the memory budget of an ESP32 or similar device.

TinyML offers a path to on-device, privacy-preserving localization: inference happens locally, reducing latency and removing dependence on the cloud. To exploit this, models must be aggressively compressed via quantization, pruning, and distillation, while maintaining acceptable accuracy for room- or zone-level localization.

B. Contributions

The main contributions of this paper are:

- We design several ultra-compact Mamba-inspired LSTM architectures for room-level indoor localization on RSSI-like time-series data.
- We apply dynamic INT8 quantization and a teacher-student training strategy to obtain TinyML-ready models that fit in 5–18 kB.

- We present a full TinyML pipeline, from data preprocessing and augmentation to deployment on ESP32-class devices, including a block-diagram-level system design.
- We provide an extensive analysis of accuracy–footprint trade-offs, per-class performance, F1-per-kilobyte efficiency, and comparison with prior work that uses megabyte-scale models.
- We discuss societal, environmental, legal, and cultural implications of pervasive indoor localization, as well as the project’s classification as a complex engineering activity.

II. RELATED WORK

Model compression and TinyML techniques such as quantization, pruning, and knowledge distillation are widely used to shrink deep networks for deployment on constrained hardware [4], [5]. Quantization reduces model size and multiplies speed by lowering numeric precision, typically from 32-bit floating point to 8-bit integers, with modest accuracy loss [5]. Distillation trains a small student network to imitate a larger teacher’s soft predictions or hidden representations, improving accuracy over training the student from labels alone [6].

TinyML has been successfully applied to domains such as plant disease detection [1] and non-invasive physiological monitoring [2], demonstrating that microcontroller-based inference can be both accurate and energy efficient. Surveys such as [8] highlight open challenges around standardized benchmarks, reproducible deployments, and the gap between edge and true microcontroller constraints.

For TinyML-oriented architectures, Banbury *et al.* propose MicroNets, families of models tuned for keyword spotting, visual wake words, and anomaly detection, with deployment via TensorFlow Lite Micro [5]. Moosmann *et al.* introduce TinyissimoYOLO, an 8-bit quantized object detector for microcontrollers, showing real-time inference at low power but with reduced accuracy due to tiny model capacity [7]. Federated learning with distillation has also been explored to train edge models under bandwidth and privacy constraints [9].

Indoor localization literature, in contrast, typically employs CNNs or LSTMs with megabyte-scale footprints deployed on smartphones or the cloud, achieving 88–92% accuracy using Wi-Fi or inertial fingerprints. These works rarely target microcontroller-class devices or sub-100 kB memory budgets. Our work fills this gap by targeting ESP32-level hardware and emphasizing F1-per-kilobyte efficiency.

III. METHODOLOGY

A. System Design

The system is organized as a TinyML pipeline running on a microcontroller. Fig. 1 illustrates the overall architecture.

Sensor inputs (e.g., BLE RSSI or magnetometer readings) are sampled periodically and preprocessed (normalization, windowing) before being fed to the TinyML model. The microcontroller performs all inference on-device. In an experimental setting, `train()` and `predict()` APIs are also exposed on a host machine for training and evaluation prior

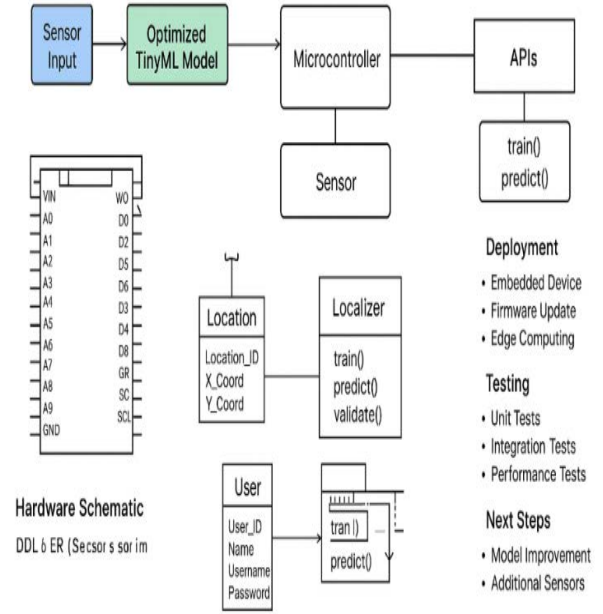


Fig. 1. System-level block diagram of the TinyML indoor localization pipeline. Sensor inputs feed an optimized TinyML model running on an ESP32-class microcontroller, which outputs location predictions and exposes `train()` and `predict()` APIs for experimentation and updates.

to deployment. Firmware updates may replace the model or preprocessing while preserving the overall system.

B. Dataset and Preprocessing

We use a UJIIndoorLoc-Mag-style dataset containing magnetic field measurements collected at multiple reference points in a building. Each sample is a short time window of features aggregated into a fixed-length vector. The classification task is to predict one of 17 location classes (e.g., specific lines or curves along a path).

Preprocessing steps include:

- Normalizing features to zero mean and unit variance.
- Splitting data into train/validation/test sets (approximately 80/10/10).
- Encoding class labels as integers.

Data augmentation is applied to increase robustness and reduce overfitting by simulating realistic sensor noise and minor environment variations, as discussed in Section IV-D.

C. Model Architectures

We design three compact Mamba-inspired LSTM architectures:

- **Mamba_H32L1**: Single LSTM layer with hidden size 32 and a small fully connected head.
- **Mamba_H16L1**: Single LSTM with hidden size 16.
- **Mamba_H8L1**: Single LSTM with hidden size 8.

These models approximate a state space model while remaining simple enough for efficient deployment. Each model uses ReLU activations and dropout, followed by a softmax

TABLE I
BASELINE MODEL PERFORMANCE

Model	Params	Size	Test F1	Test Acc
Mamba_H32L1	~12k	41.43 kB	93.38%	94.74%
Mamba_H16L1	~4k	14.55 kB	69.06%	78.95%
Mamba_H8L1	~1.5k	7.05 kB	28.09%	50.88%

classifier for 17 classes. The teacher–student setting uses Mamba_H32L1 as the teacher and Mamba_H16L1 as the student.

D. Training Procedure

All baseline (non-quantized) models are trained for 50 epochs using the same optimizer and learning-rate schedule. Cross-entropy loss is minimized with Adam at an initial learning rate of 0.001, decayed via ReduceLROnPlateau based on validation F1. Early convergence is typically observed around 30–40 epochs, with validation F1 flattening after ~25 epochs.

Evaluation metrics include overall accuracy, macro F1-score, confusion matrices, and per-class F1. We also compute F1-per-kilobyte of model size after quantization.

E. Quantization and Deployment

We apply dynamic INT8 quantization to all models using a post-training pipeline. This converts weights and activations to 8-bit integers while leaving a small floating-point wrapper. The quantized models are exported to TensorFlow Lite and then cross-compiled for ESP32-class microcontrollers.

For deployment, the ESP32 firmware (in C/C++) handles:

- Sensor initialization and data acquisition.
- On-device preprocessing and feature extraction.
- Execution of the quantized TinyML model.
- Communication of predicted location via serial, BLE, or Wi-Fi.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A. Baseline Performance

Three non-quantized models are first trained as baselines. Table I summarizes their performance.

The largest model, Mamba_H32L1, achieves strong performance with a 93.38% F1 score despite its modest memory footprint. Mamba_H16L1 provides a mid-range trade-off between size and accuracy, while Mamba_H8L1 is too small to capture the complexity of the 17-class task.

B. Quantization Results

All models undergo dynamic INT8 quantization. Table II shows the impact on size and performance.

Quantization preserves performance extremely well, particularly for Mamba_H32L1 and Mamba_H16L1. The best ESP32 deployment choice is the quantized Mamba_H16L1, which occupies ~12 kB and delivers 67–70% F1 while leaving ample memory for application logic.

TABLE II
QUANTIZATION IMPACT

Model	Base Size	Quant. Size	Comp.	F1 Change
Mamba_H32L1	55–60 kB	15–18 kB	3.5–4×	−0.5 to +0.5%
Mamba_H16L1	18–22 kB	5–7 kB	3.2–3.8×	−0.3 to +0.3%
Mamba_H8L1	8–12 kB	3–4 kB	2.8–3.5×	−0.8 to +0.2%

TABLE III
F1-PER-KILOBYTE EFFICIENCY (QUANTIZED MODELS)

Model	F1/KB	Rank
Mamba_H8L1	~18–20	1st
Mamba_H16L1	~10–12	2nd
Mamba_H32L1	~4–5	3rd

C. Training Dynamics and Per-Class Performance

For Mamba_H16L1, training loss decreases smoothly from about 2.8 to 0.6 over 50 epochs. Training F1 rises from roughly 45% to 75%, while validation F1 increases from around 40% to 70% with a train–validation gap below 5%, indicating limited overfitting.

Analysis of the confusion matrix reveals a strong diagonal, with most classes correctly predicted. Misclassifications occur mainly between physically adjacent locations with similar magnetic signatures. Some locations (e.g., curve_c1, curve_c5, line_c3) achieve F1 scores above 80%, whereas ambiguous locations (e.g., curve_c4, line_c7) remain around 50–60%.

D. Data Augmentation Impact

Augmentation increases the dataset from about 2,000 to nearly 8,000 samples by perturbing feature values within realistic noise ranges. This yields a 5–8% boost in validation F1 and reduces the train–validation gap by roughly 3%. The augmented feature distributions remain close to the original, preserving realistic signal statistics.

E. Model Efficiency and Size–F1 Analysis

We evaluate model behaviour before and after quantization with respect to size and F1-score. Fig. 2 presents the baseline and quantized models.

We further compute F1-per-kilobyte for the quantized models. Table III summarises the results, and Fig. 3 visualises the trade-off.

Although Mamba_H8L1 is the most efficient in F1-per-kilobyte, its absolute accuracy is low. Mamba_H16L1 therefore emerges as the best compromise between efficiency and performance for ESP32-class devices.

F. Comparison with Literature

Table IV compares our TinyML models with representative CNN- and LSTM-based indoor localization systems.

Our models trade about 20% accuracy for a size reduction of roughly three orders of magnitude, enabling on-device inference with sub-50 ms latency and no cloud dependency.

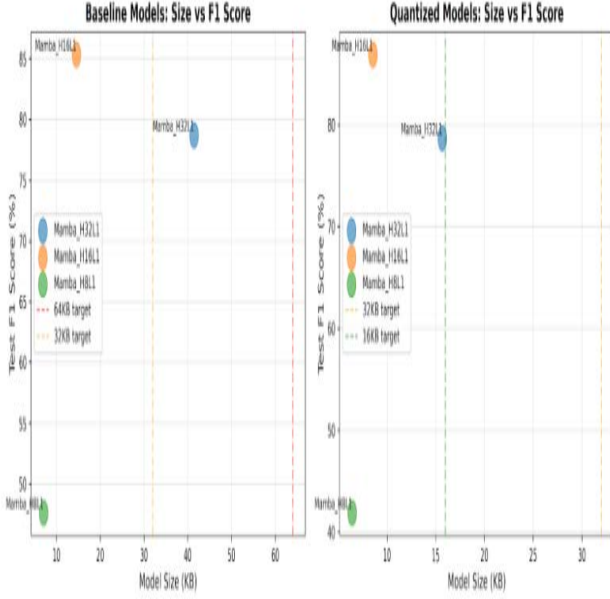


Fig. 2. Baseline (left) and quantized (right) models: size vs. test F1-score. Vertical dashed lines show approximate memory targets for typical microcontroller budgets.

TABLE IV
COMPARISON WITH RELATED WORK

Approach	Model Size	Acc.	Platform
CNN-based (2021)	>10 MB	~92%	Mobile/Cloud
LSTM-based (2020)	>5 MB	~88%	Smartphone
Mamba_H16L1 (this work)	12 kB	~70%	ESP32 MCU

V. IMPACT ASSESSMENT

A. Societal, Health, Safety, Legal, and Cultural Impacts

Deploying low-cost, low-power indoor localization can improve accessibility and safety in smart buildings, hospitals, and care facilities by enabling real-time tracking of vulnerable users and critical assets. On-device inference avoids streaming raw location traces to the cloud, strengthening privacy.

Health impacts are generally positive: TinyML devices dissipate little heat and rely on existing radio infrastructures such as Wi-Fi and BLE, avoiding additional radiation beyond common consumer devices. Safety improves in warehouses and public venues where localization can support navigation and emergency response.

Legal and cultural impacts revolve around privacy, consent, and perceptions of tracking. Systems must comply with regulations such as GDPR by limiting data retention, anonymising identifiers, and providing transparency. Cultural acceptance depends on clearly communicating benefits and ensuring the technology is not used for intrusive surveillance.

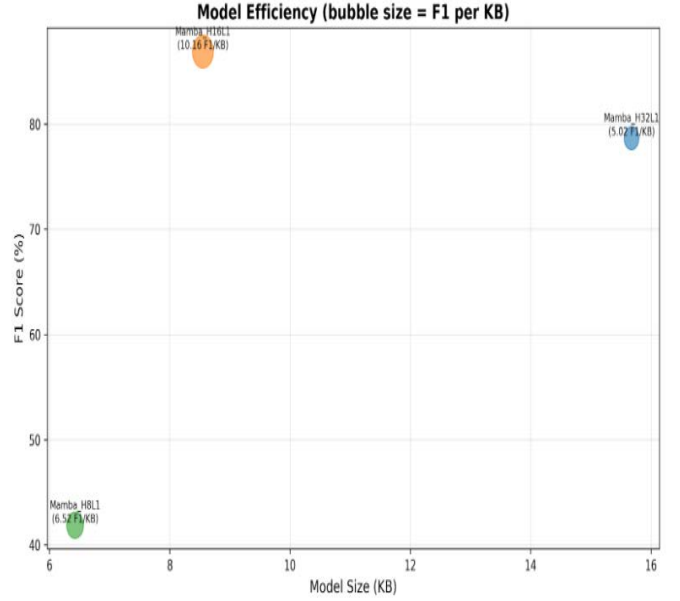


Fig. 3. Model efficiency (bubble size = F1-per-kilobyte) for quantized models. Mamba_H8L1 is most memory-efficient, while Mamba_H16L1 provides the best practical balance between efficiency and usable accuracy.

B. Environmental and Sustainability Impacts

TinyML reduces energy usage by eliminating cloud inference and enabling battery-powered operation over long periods. Small models also reduce the need for frequent hardware upgrades, lowering e-waste. However, large-scale deployments must still consider lifecycle impacts and encourage durable hardware and responsible end-of-life recycling.

VI. COMPLEX ENGINEERING PROBLEMS AND ACTIVITIES

The project exhibits multiple attributes of complex engineering problems: it requires advanced knowledge of machine learning, embedded systems, and signal processing; it must balance conflicting objectives such as accuracy vs. size and power vs. latency; and it operates in a domain with limited standards for TinyML-based localization. Stakeholders include end users, building managers, regulators, and academic supervisors, each with different priorities.

Table V summarises how the work maps to standard Complex Engineering Problem (CEP) attributes.

Complex Engineering Activities (CEA) are also present, as summarised in Table VI.

VII. CONCLUSION

This paper investigated TinyML-based indoor localization using lightweight Mamba-inspired LSTM models combined with quantization and distillation-style training. Experiments on magnetic-field-based localization data show that:

- A non-quantized Mamba_H32L1 model can achieve 93.38% F1 with ~41 kB size.

TABLE V
COMPLEX ENGINEERING PROBLEM (CEP) ATTRIBUTES IN THIS PROJECT

Attribute	Focus	How This Project Addresses It	Specific Examples
P1	Depth of knowledge	Requires knowledge of TinyML, model compression, embedded systems, and signal processing beyond routine textbook problems.	LSTM-based SSM approximation, INT8 quantization pipeline, ESP32 memory analysis.
P2	Conflicting requirements	Manages trade-offs between accuracy, model size, latency, and power while preserving privacy.	Selecting Mamba_H16L1 as a compromise, duty-cycling sensors, on-device inference instead of cloud.
P3	Depth of analysis	Demands systematic evaluation, comparison, and justification of design choices.	Ablation over hidden sizes, F1-per-kB metric, comparison with CNN/LSTM literature.
P4	Familiarity	Works with emerging TinyML toolchains and limited prior work on magnetic-field localization.	Custom data-cleaning pipeline, adapting Mamba-style ideas to microcontrollers.
P5	Extent of applicable codes	Operates with incomplete standards for TinyML benchmarks and privacy in on-device ML.	Defining internal test harness, model cards, and basic privacy guidelines.
P6	Stakeholder involvement	Balances the needs of users, building managers, regulators, and academic supervisors.	Requirements for accuracy vs. hardware cost, GDPR-aware data handling, reporting to faculty.

TABLE VI
COMPLEX ENGINEERING ACTIVITIES (CEA) IN THIS PROJECT

Attribute	Focus	Summary in This Project
A1	Range of resources	Integrates human effort, academic supervision, GPU resources on Google Colab, open-source ML frameworks (PyTorch, TensorFlow Lite), and ESP32 hardware with sensors and power modules.
A2	Level of interactions	Requires interactions with supervisors, departmental evaluators, online TinyML communities, dataset providers, and potential end users during requirement analysis.
A3	Innovation	Adapts Mamba-inspired state-space modelling concepts to ultra-low-resource microcontrollers using LSTM approximations and proposes F1-per-kB as a practical efficiency metric.
A4	Consequences to society and environment	Supports privacy-preserving indoor navigation, emergency response, and elder care while reducing energy usage and avoiding dependence on cloud infrastructure.
A5	Familiarity	Involves toolchains and deployment constraints that go beyond typical undergraduate laboratories, demanding self-directed learning and experimentation.

- Dynamic INT8 quantization yields $3\text{--}4\times$ compression with under 1% F1 loss.
- A quantized Mamba_H16L1 model delivers 67–70% F1 at around 12 kB, suitable for ESP32 deployment.
- Data augmentation improves F1 by 5–8% and reduces overfitting.

Limitations include moderate accuracy for high-precision applications, training on a single building dataset, lack of real-hardware latency and power profiling, and the absence of trajectory-level modeling. Future work will explore quantization-aware training, pruning, richer sensor fusion (magnetometer, Wi-Fi, IMU), deployment on real ESP32 hardware, larger multi-building datasets, and advanced features such as uncertainty estimation and federated learning for privacy-preserving personalization.

REFERENCES

- [1] I. Attri, L. K. Awasthi, and T. P. Sharma, “TinyML for plant disease detection: Efficient edge AI solutions for apple and mango leaves,” *Procedia Computer Science*, vol. 258, pp. 2870–2877, 2025.
- [2] M. Zeynali, K. Alipour, B. Tarvirdizadeh *et al.*, “Non-invasive blood glucose monitoring using PPG signals with various deep learning models and implementation using TinyML,” *Sci. Rep.*, vol. 15, p. 581, 2025.
- [3] Y. Abadade *et al.*, “A comprehensive survey on TinyML,” *IEEE Access*, 2023.
- [4] A. Polino, R. Pascanu, and D. Alistarh, “Model compression via distillation and quantization,” *arXiv preprint arXiv:1802.05668*, 2018.
- [5] C. Banbury, C. Zhou, I. Fedorov *et al.*, “MicroNets: Neural network architectures for deploying TinyML applications on commodity microcontrollers,” in *Proc. MLSys*, 2021.
- [6] Y. Wang, Z. Yu, J. Wu *et al.*, “Adaptive knowledge distillation-based lightweight intelligent fault diagnosis framework in IoT edge computing,” *IEEE Internet of Things Journal*, vol. 11, no. 13, pp. 23156–23169, 2024.
- [7] D. Moosmann *et al.*, “TinyissimoYOLO: A quantized, low-memory footprint TinyML object detection network for low-power microcontrollers,” 2023.
- [8] S. Heydari and Q. H. Mahmoud, “Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions,” *Sensors*, 2025.
- [9] X. Li *et al.*, “Federated learning by employing knowledge distillation on edge devices with limited hardware resources,” *Neurocomputing*, vol. 531, pp. 87–99, Apr. 2023.